

بسمه تعالی



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

فاز اول پروژه

درس سیستم های عامل

امیرعلی رحیمی

نیمسال اول ۴۰۴-۴۰۵

گزارش فاز ۱ -

این پروژه بخشی از درس سیستم‌های عامل است که بر روی سیستم عامل آموزشی XV6 کار می‌کند. XV6 یک سیستم عامل ساده شبیه‌سازی شده بر اساس Unix است که برای آموزش مفاهیم پایه‌ای سیستم عامل طراحی شده است.

ساختار XV6

- کرنل: هسته سیستم عامل که در سطح supervisor کار می‌کند
- User Space: برنامه‌های کاربردی که در سطح user اجرا می‌شوند
- System Calls: رابط بین user space و kernel space

اهداف فاز ۱

۱. آشنایی با ساختار XV6
۲. افزودن System Call جدید
۳. مدیریت پردازنده‌ها و درخت پردازنده
۴. درک مفاهیم پایه‌ای سیستم عامل

محیط توسعه

- Docker: برای ایجاد محیط یکسان توسعه
- QEMU: برای شبیه‌سازی معماری RISC-V

بخش اول:

: (System Call Counter)

هدف: ایجاد یک سیستم کال که تعداد کل سیستم کال‌های صدا زده شده از زمان بوت را برگرداند.

مراحل پیاده‌سازی:

۱. تعریف شماره سیستم کال در syscall.h
۲. ایجاد متغیر global برای شمارش در syscall.c
۳. اضافه کردن به جدول سیستم کال‌ها
۴. پیاده‌سازی تابع در sysproc.c
۵. ایجاد interface کاربر در user.h و usys.pl
۶. پیاده‌سازی برنامه sysclcnt.c
۷. ساخت یک برنامه برای تست عملکرد (test_counter.c)

مفاهیم آموزشی یادگیری شده

۱. نحوه کار سیستم کال‌ها در XV6
۲. ارتباط بین user space و kernel space
۳. مدیریت حافظه و متغیرهای global در کرنل
۴. کامپایل و build سیستم عامل

1-

```
kernel > C syscall.h > SYS_sysclcnt
syscall.h  Git Local Changes (Working Tree) - 1 of 1 change
18 18 #define SYS_mknod 17
19 19 #define SYS_unlink 18
20 20 #define SYS_link 19
21 21 #define SYS_mkdir 20
22 22 #define SYS_close 21
23 23 #define SYS_sysclcnt 22
23 24
24
```

2-

syscall.c Git Local Changes (Working Tree) - 1 of 4 changes

```
6 6 #include "proc.h"
7 7 #include "syscall.h"
8 8 #include "defs.h"
9 uint64 syscall_count = 0; //added
9 10
10 11 // Fetch the uint64 at addr from the current process.
11 12 int
```

3-

syscall.c Git Local Changes (Working Tree) - 2 of 4 changes

```
101 102 extern uint64 sys_link(void);
102 103 extern uint64 sys_mkdir(void);
103 104 extern uint64 sys_close(void);
105 extern uint64 sys_sysclnt(void); //added
104 106
105 107 // An array mapping syscall numbers from syscall.h
106 108 // to the function that handles the system call.
```

syscall.c Git Local Changes (Working Tree) - 3 of 4 changes

```
126 128 [SYS_link] sys_link,
127 129 [SYS_mkdir] sys_mkdir,
128 130 [SYS_close] sys_close,
131 [SYS_sysclnt] sys_sysclnt, //added
129 132 };
130 133
131 134 void
134 void
135 syscall(void)
136 {
137 int num;
138 struct proc *p = myproc();
139
140 num = p->trapframe->a7;
141 if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
142 // Use num to lookup the system call function for num, call it,
143 // and store its return value in p->trapframe->a0
144 p->trapframe->a0 = syscalls[num]();
145 syscall_count++; //added
146 } else {
147 printf("%d %s: unknown sys call %d\n",
148 p->pid, p->name, num);
149 p->trapframe->a0 = -1;
150 }
151 }
```

4-

sysproc.c Git Local Changes (Working Tree) - 1 of 2 changes

```
6      6 #include "spinlock.h"
7      7 #include "proc.h"
8      8 #include "vm.h"
9      9 extern uint64 syscall_count;
10     10
11     11 uint64
12     12 sys_exit(void)
```

sysproc.c Git Local Changes (Working Tree) - 2 of 2 changes

```
108    109     return xticks;
109    110 }
110    111
112    112 uint64
113    113 sys_sysclnt(void)
114    114 {
115    115     return syscall_count;
116    116 }
```

5-

user.h Git Local Changes (Working Tree) - 2 of 2 changes

```
24    25 char* sys_sbrk(int,int);
25    26 int pause(int);
26    27 int uptime(void);
27    28 int sysclnt(void);
27    29
```

user > usys.pl

usys.pl Git Local Changes (Working Tree) - 1 of 1 change

```
40    40 entry("dup");
41    41 entry("getpid");
42    42 entry("sbrk");
43    43 entry("pause");
44    44 entry("uptime");
45    45 entry("sysclnt");
45    46
```

6-

user > C sysclnt.c > main()

```
1  #include "user.h"
2
3  int main()
4  {
5      int count = sysclnt();
6      printf("%d\n", count);
7      exit(0);
8  }
```

7-test_counter:

```

user > C test_counter.c > main()
1  #include "user.h"
2
3  int main()
4  {
5      int first = sysclnt();
6      int second = sysclnt();
7      int third = sysclnt();
8      printf("First sysclnt: %d\n", first);
9      printf("Second sysclnt: %d\n", second);
10     printf("Third sysclnt: %d\n", third);
11
12     exit(0);
13 }

```

دلیل اضافه کردن فایل تست:

برای اطمینان از عملکرد صحیح سیستم‌کال `sysclnt` و بررسی اینکه بعد از دو بار پشت سر هم صدا زدن دقیقاً ۱ واحد افزایش می‌یابد، بدون تأثیر `printf` های اضافی که خودشان سیستم‌کال ایجاد می‌کنند. این تست ثابت کرد که سیستم‌کال به درستی شمارش می‌کند.

در نهایت نتیجه ی اجرای `test_counter` و `sysclnt` :

```

$ sysclnt
450
$ sysclnt
469
$ sysclnt
488
$ test_counter
First sysclnt: 511
Second sysclnt: 512
Third sysclnt: 513
$ sysclnt
589
$ sysclnt
608
$ █

```

بخش دوم:

: ptree

هدف

ایجاد یک syscall که:

- کل لیست فرآیندهای فعال سیستم را جمع‌آوری کند.
- رابطه $\text{parent} \rightarrow \text{child}$ بین فرآیندها را استخراج کند.
- یک ساختار درختی از آن‌ها بسازد.
- این ساختار را به فضای کاربر برگرداند تا برنامه ptree بتواند آن را چاپ کند.

مراحل انجام این مرحله:

۱. ساخت ساختار داده‌های لازم

در فایل جدید kernel/proc_tree.h دو ساختار تعریف شدند:

- struct proc_info
- struct proc_tree

این دو ساختار امکان ذخیره‌ی اطلاعات پایه‌ای هر فرآیند به همراه لیست فرزندانش را فراهم می‌کند.

۲. پیاده‌سازی تابع build_process_tree

در kernel/proc.c تابع build_process_tree اضافه شد

این تابع وظیفه انجام کارهای زیر را دارد:

(الف) پیمایش جدول فرآیندها (proc[]) برای هر فرآیندی که UNUSED نباشد نام، PID، PPID، state و مقداردهی اولیه لیست فرزندانش جمع‌آوری می‌شود.

(ب) ساخت رابطه $\text{parent} \rightarrow \text{child}$

با استفاده از ppidها، لیست فرزندانش برای هر فرآیند پر می‌شود.

(ج) بررسی وجود root

اگر کاربر pid ریشه را ارسال کرده باشد، بررسی می‌شود که آن pid در ساختار وجود دارد یا نه.

(د) برگرداندن نتیجه

تمام ساختار tree به user برگردانده می‌شود.

۳. اضافه کردن syscall جدید

سه مرحله اصلی انجام شد:

(الف) اضافه کردن شماره syscall در فایل kernel/syscall.h:

```
#define SYS_ptree 23
```

(ب) ثبت syscall در جدول در kernel/syscall.c:

[SYS_ptree] sys_ptree,

(ج) پیاده‌سازی sys_ptree در فایل kernel/sysproc.c :

۴. پیاده‌سازی ptree :

برنامه ptree در فضای user قرار دارد و مسئولیتش این است که:

1. یک syscall به نام ptree فراخوانی کند.
2. ساختار درخت فرآیندها را از کرنل بگیرد.
3. درخت را پردازش و چاپ کند.

در Xv6 ، تمام syscall ها باید در user/user.h اعلان شوند.

بنابراین ابتدا این تابع را اضافه می‌کنیم:

```
int ptree(int pid, void *tree);
```

این تابع فقط یک wrapper است.

وقتی کد user ptree را صدا می‌زند، در نهایت سیستم‌کال SYS_ptree فراخوانی می‌شود.

در بخش user ابتدا یک struct proc_tree تعریف می‌کنیم (همان نسخه‌ای که در header مشترک ساخته‌ایم). سپس Syscall را صدا می‌زنیم و بعد از آن درخت را چاپ می‌کنیم.

چاپ درخت چگونه انجام می‌شود؟

برای این کار یک تابع بازگشتی تعریف کردیم که سودو کد این فرآیند بدین صورت می‌باشد:

print current node

for each child:

calculate is_last = (child_index == child_count-1)

build new_prefix

print_tree(child_pid, new_prefix)

همچنین یک فایل lptree(large ptree) برای تست اجرای عملکرد صحیح ptree ساخته شده است که چند فرآیند تو در تو با استفاده از fork می‌سازد.

بخش سوم:

پیاده‌سازی Reparenting :

هدف

تغییر رفتار پیش‌فرض XV6 هنگام مرگ یک فرآیند.

رفتار پیش‌فرض: فرزندان فرآیند مرده به `init (PID 1)` داده می‌شوند.

رفتار جدید مورد نیاز پروژه: فرزندان فرآیند مرده باید به والد آن فرآیند منتقل شوند.

۱. تغییر تابع `reparent`

تابع اصلی که فرزندان یک فرآیند را به پدر جدید منتقل می‌کند، `reparent` است.

نسخه اصلی XV6 :

```
pp->parent = initproc;
```

که به صورت پیش‌فرض فرزندان یک فرآیند `kill` شده را به `initproc` ، `reparent` می‌کند

اما اگر ما این بخش را تغییر بدهیم به این حالت:

```
pp->parent = p->parent ? p->parent : initproc;
```

در این صورت با مرگ یک `process` فرزندان آن به والد `process` کشته شده `reparent` می‌شوند.

چرا این تغییر کافی است؟

به دلیل ساختار XV6 :

- تابع `kexit` قبل از فراخوانی `reparent(p)` قفل `wait_lock` را می‌گیرد.
- این یعنی `reparent` در شرایط ایمن از `race condition` اجرا می‌شود.

در ادامه تصویری از عملکرد صحیح اجرای ۳ بخش پروژه پیوست شده است:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sysctlcnt
40
$ test_counter
First sysctlcnt: 62
Second sysctlcnt: 63
Third sysctlcnt: 64
$ ptree
build_process_tree: built tree with 3 processes
=== Process Tree ===
PID: 1
├── PID: 2
│   └── PID: 5
$ lptree
=== Creating test processes ===
build_process_tree: built tree with 8 processes
ptree succeeded! Found 8 processes
PID: 1
├── PID: 2
│   ├── PID: 6
│   │   ├── PID: 7
│   │   │   ├── PID: 9
│   │   │   └── PID: 10
│   │   └── PID: 8
│   │       └── PID: 11
$ kill 7
$ ptree
$ build_process_tree: built tree with 7 processes
=== Process Tree ===
PID: 1
├── PID: 2
│   ├── PID: 13
│   ├── PID: 8
│   │   └── PID: 11
│   ├── PID: 9
│   └── PID: 10
```